Instead of expecting specific IO method calls, we can use the StringIO high-fidelity fake from the Ruby standard library. StringIO objects exist in memory, but act like any other Ruby IO object, such as an open file or Unix pipe.

You can test input-handling code by initializing a StringIO with data and letting your code read from it. Or you can test your output code by letting your code write to a StringIO, and then inspecting the contents via its string method. Here's how this test looks with a StringIO object injected into the RubyDocServer:

15-using-test-doubles-effectively/20/ruby_doc_server/spec/ruby_doc_server_spec.rb

```
require 'ruby_doc_server'
require 'stringio'

RSpec.describe RubyDocServer do
  it 'finds matching ruby methods' do
    result = get('/Array/min')

    expect(result.split("\n")).to eq [
      'Content-Type: application/json',
      '',
      '["min","min_by","minmax","minmax_by"]'
    ]
  end

  def get(path)
    output = StringIO.new
    RubyDocServer.new(output: output).process_request(path)
    output.string
  end
end
```

Now, we're setting expectations on the *contents* of the response, rather than on how it was produced. This practice results in much less brittle specs.

Wrapping a Third-Party Dependency

While high-fidelity fakes can save a lot of trouble, they can still leave your logic coupled to a third-party API. This coupling has a few downsides:

- Your code will be exposed to the complexities of a (potentially) large API.
- Changes to the dependency's interface will ripple throughout your system.
- Testing is difficult, since you can't easily and safely replace the third-party code with a double.

To avoid these pitfalls, you can wrap the dependency—that is, write your own layer that delegates to it internally. Using a wrapper (also known as a *gateway* or *adapter* when you're wrapping an API) gives you a couple of key advantages:

- You can create a small, simple interface tailored exactly to your needs.
- You can safely replace your wrapper with a pure double in your specs.